Shipment Integrity Platform

Technical Strategy, Architecture and Delivery Plan

1. Problem, Scope and Client Context

The client does not primarily have a webhook problem; it has a state-integrity problem. DHL events can arrive late, out of order, more than once, or with contradictory information. If each downstream consumer interprets those events independently, customer tracking, support and incident response can legitimately reach different conclusions. The immediate goal is therefore to establish one trusted current shipment state while preserving the full event history that explains how that state was derived.

What is in scope now

- Receive and validate DHL shipment events over an authenticated HTTPS webhook.
- Normalize partner-specific values into a stable internal event model.
- Detect duplicates, late/out-of-order events and conflicting transitions.
- Persist immutable event history and maintain a fast trusted ShipmentState projection.
- Expose current state and history to internal tracking, support and incident-response consumers.
- Provide observability, failure handling, controlled rollout, rollback and recovery.

Deliberately postponed

A customer/admin UI, additional courier partners, full event sourcing, Kafka, advanced analytics/ML and multi-region active-active deployment are intentionally deferred until business value, scale or operational evidence justifies the additional complexity.

Assumptions to validate and open questions

Question / assumption	Why it matters
Is partner and eventId guaranteed to identify one logical event? Can the same ID be resent with changed payload?	Defines idempotency and the SQL uniqueness rule.
Is occurredAt authoritative, and does DHL provide sequence/correction semantics?	Determines chronology, stale-event behaviour and tie-breaking.
What is the authoritative DHL state-transition rules and terminal-state exceptions?	The processor must not invent business semantics.
What are average/peak event rates, retry behaviour, burst duration and acceptable state-update latency?	Service Bus, processors, SQL Server and alert thresholds.
What is retention, Recovery Time Object and Recovery Point Object (RTO/RPO), webhook authentication and operational ownership requirements?	Drives security, cost, recovery and support design.

Ownership boundaries

Courier contract/credentials: client integration team; transition rules: business/product owner; API and integrity logic: application engineering; Service Bus/SQL platform: cloud/platform team; monitoring, Dead-Letter Queue (DLQ) and incidents: application operations/on-call; courier-side delivery faults: DHL via the client integration owner.

2. Proposed Production Architecture and Trade-offs

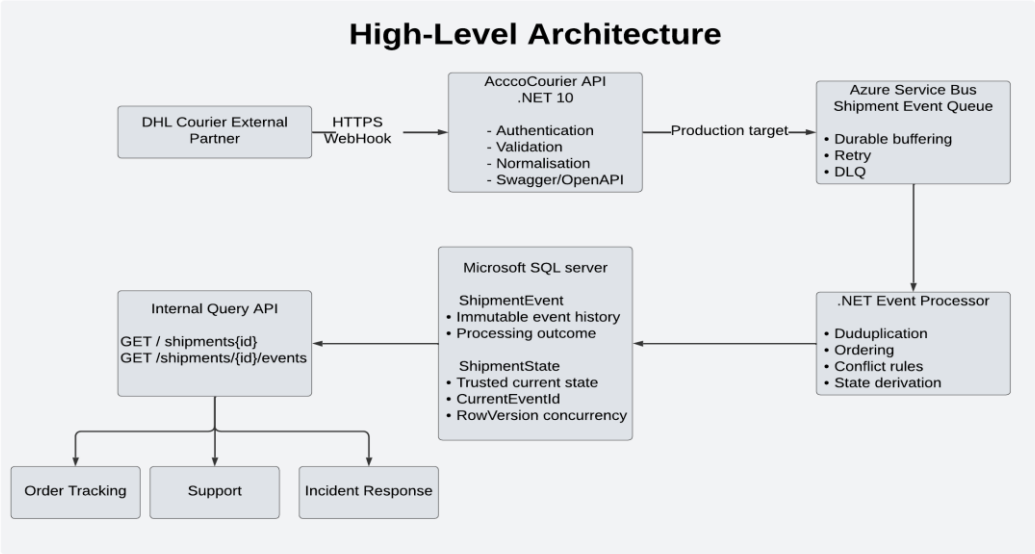

Figure 1. High-level target architecture. The executable slice uses the same domain boundaries but omits Service Bus and runs directly against local Microsoft SQL Server.

The production recommendation separates ingestion from state processing. The .NET 10 API validates and normalizes the courier request, then hands it durably to Azure Service Bus. A .NET worker applies deterministic integrity rules and updates SQL Server/Azure SQL. Event history remains the evidence; ShipmentState is the optimized operational view.

Key architecture trade-offs

Decision	Benefit	Cost / trade-off
Service Bus vs synchronous end-to-end processing	Absorbs burst and protect webhook availability from processor/SQL latency.	Adds eventual consistency, messaging operations and DLQ ownership.
Microsoft SQL Server vs Cosmos DB	Strong transactions, uniqueness, row-version concurrency and straightforward history/state queries.	Less globally elastic; scaling is more conventional.
Service Bus vs Kafka	Managed fit for durable work-queue processing, retries and poison-message isolation.	Kafka would be stronger for large replayable event-stream ecosystems if that later becomes a requirement.
History + current projection vs full event sourcing	Auditable, fast to query and simpler to operate.	Replay/versioning capabilities are intentionally more limited.
No UI in the first release	Focus effort on the highest-risk integrity problem and executable API.	Operations may later need conflict-resolution tooling.

Load and peak strategy

The brief provides no traffic figures, so sizing must be evidence-led: average/P95/P99 event rate, retry amplification, burst duration, processing time and query volume. The queue is justified by burst absorption and failure isolation, not an assumed big-data workload. Autoscaling follows queue age/depth and processing latency, with SQL capacity validated under concurrency.

3. Data Integrity, Persistence and Failure Strategy

The processor must produce the same result from the same event history and approved rules. Arrival orders are not treated as business orders. An incoming event is first validated/normalized, then classified against the existing shipment history and current projection.

Integrity decisions

Case	Rule	State effect
Duplicate	Same partner and eventId. Application checks early; SQL UNIQUE(Partner, EventId) is the final concurrency-safe guard.	No repeated business effect.
Out of order / stale	Incoming occurredAt is older than the timestamp represented by the trusted projection.	Retain the event; do not regress current state.
Conflict	A newer event violates the approved transition matrix, or equal timestamps claim different statuses.	Retain + flag; trusted state unchanged.
Applied	The event is chronologically valid and the transition is allowed.	Persist the event and update current state atomically.

SQL Server integrity boundary

The core persistence model is intentionally small: Shipment, ShipmentEvent and ShipmentState. ShipmentEvent stores the immutable courier evidence and processing outcome; ShipmentState stores the trusted current projection. Production persistence enforces foreign keys, UNIQUE(Partner, EventId), a transaction around event insert + state update, and ROWVERSION/optimistic concurrency for competing processors. A duplicate-key race is interpreted as an idempotent duplicate rather than a system failure.

Failure and recovery behaviour

- Do not acknowledge a webhook as durably accepted until the queue/message boundary succeeds.
- Transient workers or SQL failures are retried safely because processing is idempotent.
- After a bounded retry threshold, isolate the message in the Dead-Letter Queue and alert an operational owner.
- Do not delete historical events during recovery. Stop harmful processing, preserve evidence, then rebuild/correct the projection only after the defect is understood.
- Unknown statuses or structurally invalid payloads must not silently change trusted state; they should be rejected or quarantined according to the agreed courier contract.

Observability

The operational dashboard should show webhook success/error rate, queue depth and oldest-message age, P95 event-to-state latency, duplicate/stale/conflict rates, DLQ count and SQL health. Structured logs should include ShipmentId, EventId, Partner and processing outcome. Alerts must have an actionable threshold and a named owner rather than simply reporting infrastructure noise.

Security

Webhook authentication/signature verification, TLS, least privilege database access and externalized secrets are production requirements. Local development may use appsettings. Development.json/User Secrets; production secrets should be injected from Azure Key Vault or the client-approved equivalent. No real credentials belong in source control or Docker images.

4. Delivery, Testing, CI/CD and Release Control

Pragmatic phased delivery

Phase	What ships / evidence
1. Discovery & refinement	Confirm courier semantics, transition matrix, security, load assumptions, ownership and acceptance examples.
2. Thin executable slice	.NET 10 API and C# domain processor and ADO.NET, local MSSQL, Swagger and unit/integration tests proving APPLIED/DUPLICATE/STALE/CONFLICT.
3. Production integration	Service Bus, production SQL persistence, authentication, concurrency handling, query endpoints and integration tests.
4. Hardening	Observability, security review, failure/performance testing, runbook, release/rollback rehearsal.
5. Controlled production	Staging -> limited traffic -> full traffic using explicit go/no-go gates.

Scrum and refinement

Work should be refined from business failure scenarios rather than infrastructure tasks. A backlog item is ready when acceptance criteria, unknowns, test evidence, security/retry/observability implications and dependencies are understood. Sprints should deliver the smallest vertical slice that can be demonstrated end to end; review working integrity scenarios, then feed learning into the next refinement cycle.

Test plan

Level	Key validation	Exit evidence
Unit	Normal transition, duplicate, stale/out-of-order, same-time conflict, invalid transition.	Deterministic APPLIED / DUPLICATE / STALE / CONFLICT.
API	Valid/invalid DHL payloads, status codes and serialization.	Contracts behave consistently and rejects invalid input safely.
Integration	ADO.NET and MSSQL, unique Partner/EventId, transactions and history/state queries.	History and current state remain consistent.
Concurrency	Same shipment/event processed concurrently.	No duplicate event and no lost state update.
Failure / recovery	SQL unavailable, worker failure, retry and DLQ.	No silent loss; failure is observable and recoverable.
Performance / smoke	Normal load, burst/backlog recovery; health, webhook, state and history after deploying.	Latency/queue age within target and critical path succeeds.

Critical acceptance scenario: when OUT_FOR_DELIVERY is current and an older IN_TRANSIT event arrives, the event remains queryable in history, but the trusted current state does not regress.

CI/CD and release control

Recommended pipeline: PR -> restore/build .NET 10 -> unit tests -> MSSQL integration tests -> security/quality checks -> publish a versioned immutable artifact/container -> deploy staging -> smoke tests -> approval gate -> production -> health checks and monitoring. Database changes should use backward-compatible expand/contract techniques where practical, so application rollback remains possible.

Release and rollback gates

Development requires passing automated tests; integration requires webhook/SQL/messaging/security validation; staging requires production-like failure/performance tests, dashboards and runbook; limited production requires stable integrity/DLQ metrics; full release requires business/technical go-no-go, on-call ownership and rollback readiness. If a bad release changes state incorrectly: stop consumers, preserve event history, redeploy the previous known-good version, validate the projection, then replay/rebuild only after the faulty rule is understood.

5. Minimum Production Slice, Risks, Success Signals and Technical Debt

Minimum credible first production slice

One courier (DHL); authenticated webhook; durable queue; deterministic normalization and integrity processing; Microsoft SQL Server event history and current-state projection; minimal query API; automated tests; structured observability; retry/DLQ; release gates and documented rollback/recovery. This is enough to create a reliable operational view without building a broader logistics platform.

Executable slice submitted

The executable proof deliberately implements the highest-risk domain behaviour using .NET 10, ASP.NET Core, Swagger/OpenAPI, ADO.NET (Microsoft.Data.SqlClient) and local Microsoft SQL Server. It exposes POST /api/webhooks/dhl, GET /api/shipments/{id}, GET /api/shipments/{id}/events and /health, with xUnit/integration tests. The API can run directly over HTTPS or as a Docker container; Docker Compose is not required in the current local setup because SQL Server remains a host-managed dependency. Service Bus remains the production target boundary rather than faked into the thin slice.

Top risks and mitigations

Risk	Mitigation
Wrong or ambiguous courier transition rules	Client/business sign-off; table-driven rules; exhaustive transition tests.
Duplicate or concurrent processing	UNIQUE(Partner, EventId), transaction boundary and ROWVERSION concurrency.
Traffic burst / partner retry storm	Service Bus buffering, worker scaling and queue-age alerts.
Poison or invalid event	Bounded retries, DLQ/quarantine, structured reason and named owner.
Bad release corrupts the projection	Preserve history, stop processing, rollback app, validate and rebuild projection safely.
Untrusted/compromised webhook	Partner authentication/signature validation, secret rotation and least privilege.

Success signals

- No duplicate-driven state changes and no trusted-state regressions caused by late events.
- Every accepted event is classified as APPLIED, DUPLICATE, STALE or CONFLICT; no silent loss.
- P95 event-to-current-state latency remains within the client-approved SLA and queue backlog recovers predictably.
- Conflict and DLQ rates are visible, owned and within agreed thresholds.
- Support can explain the current state from retained history, and the projection can be recovered after rollback.

Controlled technical debt / deferred improvements

Deferred item	Reason / review trigger
Single courier implementation	DHL is the only current partner; introduce adapters when a second courier is onboarded.
Transition rules in application code	Keeps the initial policy explicit/testable; externalize if business changes become frequent.
No operational conflict UI	Not required to prove integrity; add if conflict volumes create support overhead.
Limited replay/reconciliation tooling	Add controlled replay/rebuild capability when operational use demonstrates the need.
Docker Compose	Deferred while SQL Server remains host-managed; introduce when multiple local dependencies are containerized.

Technical debt should be recorded with an owner, rationale and review trigger. Security, integrity constraints, testing, observability and rollback readiness are production requirements, not optional debt.